

Calcular el orden de complejidad de:

1. En un array de n elementos se almacenan los números de 1 a $n+1$, excepto uno de ellos. Encuentre cuál falta en un orden como máximo de $O(n)$.
2. Se dice que un número n es triangular se puede expresar como la suma de los j primeros números. Determinar si un número n es triangular o no.
- 3.

```
i = 1
j = 1
while i ≤ n and j ≤ n
    if a[i, j + 1] < a[i + 1, j]
        j = j + 1
    else
        i = i + 1
    endif
endwhile
```

4.

```
for i = 1 to n
    for j = 1 to n
        suma = 0
        for k = 1 to n
            suma = suma + a[i, k] * b[k, j]
        endfor
        c[i, j] = suma
    endfor
endfor
```

5.

```
p = 0
for i = 1 to n
    p = p + i * i
    for j = 1 to p
        write (a[p, j])
    endfor
endfor
```

6.

```
Contar (izq, der: indices): integer
  if izq  $\geq$  der
    return 0
  elsif a[izq] < a[der]
    return (Contar (izq*2, der) + 1)
  else
    return (Contar (izq*2, der))
  endif
```

7.

```
i = 1
j = n
while i < j
  if a[i] < a[j]
    i = i * 2
  else
    j = j / 2
  endif
endwhile
```

8.

```
procedure Primero (n: integer);
var
  i: integer;
  acum: real;
begin
  if n>0 then begin
    acum:= 0;
    for i:= 1 to 2 do
      acum:= acum + Segundo (n);
    write (acum);
  end;
end;
```

```
function Segundo (x: integer): real;
var
  j, k: integer;
begin
  k:= 0;
  for i:= 1 to x do
    k:= k+random(100);
    Primero (x div 2);
    Primero (x div 2);
    Segundo:= k/x;
  end;
```

9.

```
procedure dos (a: array [1..n] of integer; x, y: integer)
  if x  $\neq$  y
    if 2*a[x] < a[y]
      dos (a, x, y div 2)
    else
      dos (a, 2x, y)
    endif
  endif
```

Suponer que los datos del array **a** están ordenados de manera creciente.

10.

```
PROCEDIMIENTO inserción(v : VECTOR(ENTERO), tamaño : NATURAL) ES
  i, j : ENTERO;
  PARA i DESDE 0 HASTA tamaño - 2 HACER
    intercambia(v[i], v[posMinimo(v, i, tamaño-1)])
  FINPARA
FINPROCEDIMIENTO

FUNCION posMinimo(v : VECTOR(ENTERO), inicio, fin : NATURAL) ES
  i, result: ENTERO;
  result := inicio;
  PARA i DESDE inicio+1 HASTA fin HACER
    SI (v[i] < v[result]) ENTONCES
      result := i;
    FINSI
  FINPARA
  DEVOLVER result;
FINFUNCION
```

11.

```
PROCEDURE invierte (A: array [0..n, 0..n] of integer);
var
  i, j: integer;
begin
  for i:= 0 to n-1 do
    for j:= i+1 to n do
      A[i, j]:= 2*A[j, i]+A[i, j];
    end;
  end;
```

12. Suponga que “n” es el tamaño del arreglo “a”.

```
max = 0
for i = 1 to n
  cont = 1
  j = i + 1
  while a[i] ≤ a[j]
    j = j + 1
    cont = cont + 1
  endwhile
  if cont > max
    max = cont
  endif
endfor
```

14.

```
PROCEDIMIENTO intercambia(REF x : ENTERO, REF y : ENTERO) ES
  aux : ENTERO;
  aux := x;
  x := y;
  y := aux;
FINPROCEDIMIENTO
```

15.

```
FUNCION max(x : ENTERO, y : ENTERO) : ENTERO ES

  result : ENTERO;
  SI (x >= y) ENTONCES
    result := x;
  SINO
    result := y;
  FINSI
  DEVOLVER result;
FINFUNCION
```

16.

```
FUNCION aSaber(v : VECTOR(ENTERO), n: ENTERO) : ENTERO ES
  i, result : ENTERO;
  result := 0;    i := tamaño - 1;
  MIENTRAS (i >= 0) HACER
    result := result + v[i];
    i := i - 1;
  FINMIENTRAS
  DEVOLVER result;
FINFUNCION
```

17.

```
FUNCION aSaber(v : VECTOR(ENTERO), n: ENTERO) : ENTERO ES
  i, result : ENTERO;
  result := 0; i := tamaño - 1;
  MIENTRAS (i >= 0) HACER
    result := result + v[i];
    i := i - 2;
  FINMIENTRAS
  DEVOLVER result;
FINFUNCION
```

18. La función Intercambia sólo son procesos de asignación para colocar el valor de una variable en otra con ejecución $O(1)$

```
PROCEDIMIENTO burbuja(v : VECTOR(ENTERO), tamaño : NATURAL) ES
  i, j : ENTERO;
  PARA i DESDE 2 HASTA tamaño - 1 HACER
    PARA j DESDE 0 HASTA tamaño - i HACER
      SI (v[j] > v[j+1]) ENTONCES
        intercambia(v[j],v[j+1]);
      FINSI
    FINPARA
  FINPARA
FINPROCEDIMIENTO
```

19.

```
PROCEDIMIENTO inserción(v : VECTOR(ENTERO), tamaño : NATURAL) ES
  i, j : ENTERO;
  PARA i DESDE 0 HASTA tamaño - 2 HACER
    min : ENTERO; min := i;
    PARA j DESDE i+1 HASTA tamaño - 1 HACER
      SI (v[j] < v[min]) ENTONCES
        min := j;
      FINSI
    FINPARA
    intercambia(v[i],v[min])
  FINPARA
FINPROCEDIMIENTO
```

20.

```
int sqrt(int n) {
  for (int guess = 1; guess * guess < n; guess++) {
    if (guess * guess == n) return guess;
  }
  return -1;
}
```

```
int[] copyArray(int[] array) {  
    int[] copy = new int[0];  
    for (int value : array) {  
        copy = appendToNew(copy, value);  
    }  
    return copy;  
}  
  
int[] appendToNew(int[] array, int value) {  
    int[] bigger = new int[array.length + 1];  
    for (int I = 0; I < array.length; I++) {  
        bigger[I] = array[I];  
    }  
    bigger[bigger.length - 1] = value;  
    return bigger;  
}
```

21.

```
int sumDigits(int n) {  
    int sum = 0;  
    while (n != 0) {  
        sum += n % 10;  
        n /= 10;  
    }  
    return sum;  
}
```

22.

23.

```
public class SortByValue {
    public static void sortByValue(int[] array) {
        int count = 1;
        while (count < array.length) {
            int swapCount = 0;
            for (int i = 0; i < array.length - count; i++) {
                if (array[i] > array[i + 1]) {
                    swap(array, i, i + 1);
                    swapCount++;
                }
            }
            count++;
        }
    }

    private static void swap(int[] array, int index1, int index2) {
        int temporaryValue = array[index1];
        array[index1] = array[index2];
        array[index2] = temporaryValue;
    }

    public static void main(String[] args) {
        int[] array = {4, 2, 8, 6, 5};
        sortByValue(array);

        // Imprimir el arreglo ordenado
        for (int value : array) {
            System.out.print(value + " ");
        }
    }
}
```

En los siguientes ejercicios, escriba el algoritmo respectivo para solucionar la situación planteada y calcule su complejidad $T(n)$ para el peor caso.

Además para cada situación determine cuál es el peor caso y el mejor caso (si los llegase a haber). En caso contrario (cuando no haya ni mejor ni peor caso, escriba una justificación del porqué no existen)

Determine el Big-O y el Ω

1. Dado un número n , encuentra dos pares que puedan representar el número como la suma de dos cubos. En otras palabras, encuentre dos pares (a, b) y (c, d) de tal manera que dado el número n puede ser expresado como: $n = a^3 + b^3 = c^3 + d^3$

Ejemplo:

Entrada: $N = 1729$

Salida: $(1, 12)$ y $(9, 10)$

Explicación:

$$1729 = 1^3 + 12^3 = 9^3 + 10^3$$

Entrada: N = 4104

Salida: (2, 16) y (9, 15)

Explicación:

$$4104 = 2^3 + 16^3 = 9^3 + 15^3$$

Entrada: N = 13832

Salida: (2, 24) y (18, 20)

Explicación:

$$13832 = 2^3 + 24^3 = 18^3 + 20^3$$

2. Dados dos números a y b donde $1 \leq a \leq b$, encuentre el número de cuadrados perfectos entre a y b (a y b inclusive).

Ejemplo:

Entrada: a = 3, b = 8

Salida: 1

El único perfecto en el rango dado es 4.

Entrada : a = 9, b = 25

Salida: 3

Los tres cuadrados en el rango dado son 9,

16 y 25

3. Dado un entero n en el intervalo [1, 1000000000], genere un triplete de Pitágoras que incluyen como uno de sus lados si y sólo si, es posible obtenerlo. Los "tripleta de Pitágoras" son soluciones

enteras del Teorema de Pitágoras, es decir, satisfacen la ecuación
$$a^2 + b^2 = c^2$$

Ejemplo:

Si se digita n=122 Los números serían: a=22, b=120 y c=122

Si se digita n=22 Los números serían: a=22, b=120 y c=122

Si se digita n=120 Los números serían: a=22, b=120 y c=122

4. Dados x , n (enteros positivos) calcula x elevado a la potencia n (x^n)
5. Un computador con ALU (Aritmetich Logic Unit) limitada, solo realiza sumas y restas. Dados m y n , calcular el residuo que se obtiene al dividir m entre n . (En este caso la complejidad es función de las dos variables (m , n)).
6. Dado un array de enteros de n posiciones y un valor x determinar cuántas veces aparece x en el array.
7. Dado un array de enteros de n posiciones, calcular la suma de los elementos en las posiciones pares.
8. Dado un array bidimensional de $n \times n$ posiciones de enteros calcular la suma de todos sus elementos.
9. Dado un array bidimensional de $n \times n$ posiciones de enteros, determinar si la matriz es simétrica o no.
10. Dado un array bidimensional de $n \times n$ posiciones de enteros, calcular la suma de los elementos ubicados sobre la diagonal principal.
11. Dado un array bidimensional de $n \times n$ posiciones de enteros, calcular la suma de los elementos ubicados sobre la diagonal secundaria.
12. Dado un array de enteros de n posiciones, invertirlo sobre sí mismo.
13. Dado un array de enteros de n posiciones, determinar si está ordenado ascendentemente.
14. Una array de n posiciones contiene los números enteros desde 0 hasta n excepto uno de ellos. Los valores no tienen ningún orden preestablecido. Encuentre el valor faltante.
15. Dados dos array de n posiciones A y B que contienen elementos enteros no repetidos en cada array, construya un tercer array con aquellos valores comunes a los dos (Intersección).
16. Se quiere escribir un programa para determinar si un número es PoliDivisible. Un número es considerado PoliDivisible si:
 - a. Es divisible por su longitud.
 - b. Al quitarle sucesivamente el dígito más a la derecha el número que queda sigue siendo PoliDivisible.

Recuerde que, matemáticamente, la longitud de un número corresponde a la cantidad de sus dígitos:

- La longitud de 8, 7 y 9 es 1.
- La longitud de 15, 76 y 38 es 2.
- La longitud de 872, 101 y 935 es 3.
- La longitud de 31465, 11112 y 19746 es 5.
- La longitud de 2147483647 es 10.

A continuación, algunos ejemplos de números que son PoliDivisibles:

número	longitud	residuo de la división
9816	4	0
981	3	0
98	2	0
9	1	0
0		

número	longitud	residuo de la división
98125	5	0
9812	4	0
981	3	0
98	2	0
9	1	0
0		

número	longitud	residuo de la división
981654	6	0
98165	5	0
9816	4	0
981	3	0
98	2	0
9	1	0
0		

número	longitud	residuo de la división
381654729	9	0
38165472	8	0
3816547	7	0
381654	6	0
38165	5	0
3816	4	0
381	3	0
38	2	0
3	1	0
0		

Su algoritmo recibe como entrada un número tipo long y genere como salida un valor lógico boolean indicando si el número de la entrada es PoliDivisible

17. Proporcione un análisis del tiempo de ejecución (notación Big-Oh) para cada uno de los siguientes 4 fragmentos de programa. Tenga en cuenta que el tiempo de ejecución corresponde aquí al número de veces que se ejecuta la operación `sum++`. `sqrt` es la función que devuelve la raíz cuadrada de un número dado.

```
(a) sum = 0;
    for(i=0; i<sqrt(n)/2; i++)
        sum++;
    for(j=0; j<sqrt(n)/4; j++)
        sum++;
    for(k=0; k<8+j; k++)
        sum++;

(b) sum = 0;
    for(i=0; i<sqrt(n)/2; i++)
        for(j=i; j<8+i; j++)
            for(k=j; k<8+j; k++)
                sum++;

(c) sum = 0;
    for(i=1; i<2*n; i++)
        for(j=1; j<i*i; j++)
            for(k=1; k<j; k++)
                if (j % i == 1)
                    sum++;

(d) sum = 0;
    for(i=1; i<2*n; i++)
        for(j=1; j<i*i; j++)
            for(k=1; k<j; k++)
                if (j % i)
                    sum++;
```

A partir del análisis anterior:

- Si se necesitan 10 ms para ejecutar el programa (b) para $n = 100$, ¿cuánto tardará en ejecutarse para $n = 400$?
- Si se necesitan 10 ms para ejecutar el programa (a) para $n = 100$, ¿qué tamaño de problema se puede resolver en 40 ms?

A partir de la ley de Amdahl:

18. Un programa tarda 100 segundos en correr. El programa pasa 80 segundos en un procedimiento. ¿Qué tanto se debe mejorar ese procedimiento para que el programa tenga una mejora? De un valor de la mejora y demuestre a través de un dato en concreto.
19. Supongamos que tenemos un programa que tiene un tiempo de ejecución de X segundos y nos piden mejorar una parte del programa que se demora 20sg en ejecución y con la mejora esta parte del programa se ejecutará en solo $X/2$ sg. Sabiendo que el tiempo nuevo de ejecución es de 49 seg, ¿Cuál era el tiempo inicial de ejecución del programa?
20. Una máquina sólo consume el 50% del tiempo ejecutando un programa X , si ese programa tiene una mejora de 100 veces, ¿Cuál es la aceleración de esa mejora?
21. Supongamos que tenemos un programa de procesamiento de datos que consta de tres partes: A, B y C. La parte A representa el 30% del trabajo total y no se puede paralelizar, mientras que la parte B y la parte C, que representan el 50% y el 20% del trabajo respectivamente, son paralelizables.
 - Calcule la velocidad de ejecución mejorada si se optimizan las partes B y C para ejecutarse el doble de rápido.
 - Determine la aceleración máxima posible que se puede lograr en este programa.

- ¿Cuál sería el impacto en la velocidad de ejecución si se lograra paralelizar la parte A en un 50%?
22. -Suponer una mejora 10 veces más rápida que la máquina original, pero sólo se utiliza el 40% del tiempo. ¿Cuál es la aceleración que genera dicha mejora?
23. Si se busca mejorar el rendimiento de un equipo de cómputo mediante la introducción de un nuevo componente de video para el procesador que realice las operaciones en la mitad del tiempo:
- Determinar la aceleración considerando que el 80% del trabajo del programa se dedica a operaciones de vídeo
 - Calcular el nuevo tiempo de ejecución del programa una vez aplicada la mejora, si originalmente tardaba 30 segundos sin dicha optimización.